

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Carrera de Ingeniería de Software

Asignatura: Aplicaciones Web

Proyecto BIOPET

Sistema Integral de Gestión Veterinaria

Entrega 1B — Módulo de Autenticación JWT y Acceso a Datos

Integrantes:

Beltrán Montiel Fred Adrián C.I. 1250300876

Mariscal Cabrera Jaime Josué C.I. 1250710835

Taípe Mora Zaida Melissa C.I. 0604907956

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Repositorio:

<https://github.com/JirachinG19Stdio/PFC--VET-ENTR1B>

Período Académico: 2025 – 2026

Quevedo — Los Ríos — Ecuador

Enero de 2026

Resumen

Se desarrolló la Entrega 1B del Proyecto Final de Carrera BIOPET, un sistema integral de gestión para clínicas veterinarias, materializando su primer módulo de código funcional. El problema abordado fue la necesidad de una capa de autenticación y autorización segura que proteja la información de usuarios y pacientes antes de construir el resto del sistema. Para ello se implementó un backend REST con Java 21 y Spring Boot 3.2, empleando Spring Security 6 y la biblioteca `jjwt` 0.12 para una autenticación *stateless* basada en JSON Web Tokens firmados con HS256, con BCrypt de factor 12 para el resguardo de contraseñas. El acceso a datos se resolvió con Spring Data JPA e Hibernate sobre PostgreSQL 16, versionando el esquema con migraciones de Flyway para garantizar reproducibilidad y eliminar la concatenación manual de SQL. La revocación de sesiones se implementó mediante una lista negra de identificadores de token (JTI) en Redis con expiración automática. Se construyó el CRUD completo de la entidad Mascota y se documentaron seis controles de seguridad alineados con el OWASP Top 10. Como resultados concretos, se obtuvo un *access token* con vigencia de una hora, tiempos de respuesta del endpoint de inicio de sesión del orden de 230 milisegundos y una cobertura de cinco pruebas unitarias automatizadas en estado satisfactorio. Todo el sistema se orquestó con Docker Compose en cuatro servicios verificables con un único comando.

Palabras clave: JWT, Spring Boot, Spring Security, OWASP, Redis, Spring Data JPA, Flyway, Docker, autenticación *stateless*.

Índice

Resumen	1
1. Introducción	3
1.1. Contexto y motivación	3
1.2. Objetivos de la entrega	3
1.3. Alcance	3
1.4. Estructura del documento	3
2. Marco Teórico	4
2.1. Autenticación <i>stateless</i> frente a <i>stateful</i>	4
2.2. Spring Security 6 y el modelo de filtros	4
2.3. Mapeo objeto-relacional con JPA e Hibernate	4
2.4. Migraciones de base de datos con Flyway	4
2.5. OWASP y seguridad de JWT	4
3. Diseño del Sistema	5
3.1. Actualización del modelo C4 (Nivel 2 — Contenedores)	5
3.2. Diagrama de clases UML	5
3.3. Diagrama de secuencia: flujo de autenticación JWT	7
3.4. Script Flyway de migración inicial	7
4. Implementación	10
4.1. Estructura del repositorio (monorepo)	10
4.2. Configuración de Spring Boot	11
4.3. Implementación del filtro JWT	12
4.4. Endpoints implementados	12
4.5. Estructura del JWT emitido	13
4.6. Implementación Angular: interceptor JWT	13
5. Pruebas	14
5.1. Pruebas unitarias del servicio de autenticación	14
5.2. Pruebas de integración con Postman	15
5.3. Métricas de rendimiento	18
6. Seguridad Implementada	19
7. Docker Compose y Despliegue	20
8. Historial de Git	22
9. Conclusiones	22
Referencias	23

1. Introducción

1.1. Contexto y motivación

La operación diaria de una clínica veterinaria involucra el registro de mascotas, sus dueños, citas, historiales clínicos y facturación, procesos frecuentemente gestionados en hojas de cálculo o en papel, lo que provoca pérdida de información y demoras en la atención. BIOPET busca centralizar esta gestión en una plataforma web segura. Dado que el sistema manejará datos sensibles de usuarios y pacientes, la base de todo el proyecto es una capa de autenticación y autorización robusta; un esquema mal diseñado expondría el sistema a accesos indebidos, robo de credenciales y escalamiento de privilegios. Por ello, esta entrega construye dicho cimiento de seguridad antes que cualquier otra funcionalidad. [12]

1.2. Objetivos de la entrega

1. **Implementar** la autenticación *stateless* con JWT, cubriendo el registro, el inicio de sesión, el cierre de sesión con lista negra en Redis y la renovación de token. [13]
2. **Construir** el CRUD completo de la entidad Mascota mediante Spring Data JPA, gestionando el esquema con migraciones versionadas de Flyway.
3. **Aplicar y validar** controles de seguridad alineados con el OWASP Top 10 y verificar el comportamiento del sistema con pruebas unitarias automatizadas.
4. **Demostrar** el despliegue reproducible del sistema completo con Docker Compose y evaluar el impacto de la caché en el rendimiento.

1.3. Alcance

Esta entrega comprende el backend funcional (autenticación, autorización por roles y CRUD de la entidad Mascota), la documentación de diseño (diagramas C4, de clases, entidad-relación y de secuencia), la batería de pruebas, la colección de Postman, la configuración de despliegue con Docker y el control de versiones. La interfaz de usuario en Angular se entrega como esqueleto inicial (interceptor, *guard* y servicio de autenticación), reservando su desarrollo completo para la Entrega 2.

1.4. Estructura del documento

El presente informe se organiza de la siguiente manera: el marco teórico sustenta las decisiones técnicas con referencias académicas; el diseño del sistema presenta los diagramas C4, de clases, de secuencia y el modelo de datos; la implementación describe la estructura del repositorio, la configuración, el filtro JWT, los endpoints y el cliente Angular; la sección de pruebas reúne las pruebas unitarias, de integración y las métricas; finalmente se documentan la seguridad implementada, el despliegue con Docker, el historial de Git, las conclusiones por objetivo y las referencias.

2. Marco Teórico

2.1. Autenticación *stateless* frente a *stateful*

En un esquema *stateful*, el servidor almacena la sesión de cada usuario, lo que dificulta el escalado horizontal. El enfoque *stateless* con JSON Web Token traslada el estado al propio token: un objeto JSON firmado digitalmente que el servidor valida en cada solicitud sin guardar estado [3]. Este modelo es idóneo para arquitecturas SPA/API como la de BIOPET, donde el cliente Angular consume una API REST. JWT se enmarca en los flujos de autorización definidos por OAuth 2.0 [4], y su principal desafío —la revocación anticipada— se aborda en este trabajo con una lista negra. La literatura indica que la elección del algoritmo de firma incide en el rendimiento y la seguridad [7], por lo que se optó por HMAC-SHA256 con un secreto de al menos 256 bits.

2.2. Spring Security 6 y el modelo de filtros

Spring Security estructura la seguridad como una `SecurityFilterChain`: una cadena de filtros que intercepta cada solicitud [2]. El `AuthenticationManager` coordina la autenticación, delegando en un `UserDetailsService` que recupera al usuario desde la base de datos y un `PasswordEncoder` que verifica las credenciales. En este proyecto, un filtro propio que extiende `OncePerRequestFilter` valida el JWT, consulta la lista negra y puebla el `SecurityContext`.

2.3. Mapeo objeto-relacional con JPA e Hibernate

Frente al uso de JDBC puro —verboso y propenso a errores de concatenación de SQL—, Spring Data JPA sobre Hibernate ofrece repositorios genéricos, consultas derivadas del nombre del método, paginación y consultas parametrizadas que previenen la inyección SQL [1]. Hibernate gestiona el ciclo de vida de las entidades y el caché de primer nivel, reduciendo el código repetitivo de la capa de persistencia. [11]

2.4. Migraciones de base de datos con Flyway

Flyway versiona el esquema de la base de datos mediante scripts SQL aplicados en orden y de forma reproducible en todos los entornos. Se distinguen las migraciones versionadas, con prefijo `V1__`, que se ejecutan una sola vez y nunca deben modificarse tras aplicarse, de las migraciones repetibles, con prefijo `R__`, que se reejecutan cada vez que cambia su contenido (útiles para vistas o funciones). Este versionado garantiza que desarrollo, pruebas y producción compartan exactamente la misma estructura de datos.

2.5. OWASP y seguridad de JWT

El OWASP Top 10 sintetiza los riesgos más críticos en aplicaciones web [5]. La categoría A07 (Fallas de identificación y autenticación) es central en esta entrega: recomienda

tokens de vigencia corta, mecanismos de revocación y almacenamiento seguro del token en el cliente. En línea con ello, el *access token* se conserva en memoria del cliente Angular —nunca en `localStorage`, vulnerable a XSS— y el *refresh token* se gestiona como cookie `HttpOnly`. Estas decisiones, junto con BCrypt y las cabeceras de seguridad, se detallan en la Sección 6.

3. Diseño del Sistema

3.1. Actualización del modelo C4 (Nivel 2 — Contenedores)

El diagrama de contenedores actualiza el de la Entrega 1A incorporando los nuevos contenedores. El frontend Angular (puerto 4200) se comunica por HTTP/REST con el backend Spring Boot (puerto 8080), que a su vez accede a PostgreSQL (puerto 5432) vía JDBC y a Redis (puerto 6379) para la caché y la lista negra de tokens.

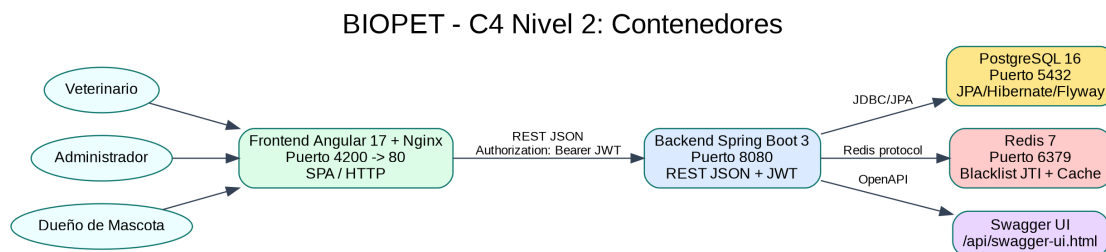


Figura 1: Vista de contenedores del sistema BIOPET (Modelo C4, Nivel 2).

3.2. Diagrama de clases UML

El diagrama de clases representa la estructura completa del backend, agrupada en paquetes (`entity`, `dto`, `service`, `repository`, `controller`, `security`, `config`), con las clases de entidad anotadas con `@Entity`, las interfaces de repositorio que extienden `JpaRepository`, los servicios con sus dependencias inyectadas por constructor, los DTO como `record` sin anotaciones JPA y las clases de seguridad (`JwtService`, `JwtAuthenticationFilter`, `SecurityConfig`, `UserDetailsServiceImpl`).

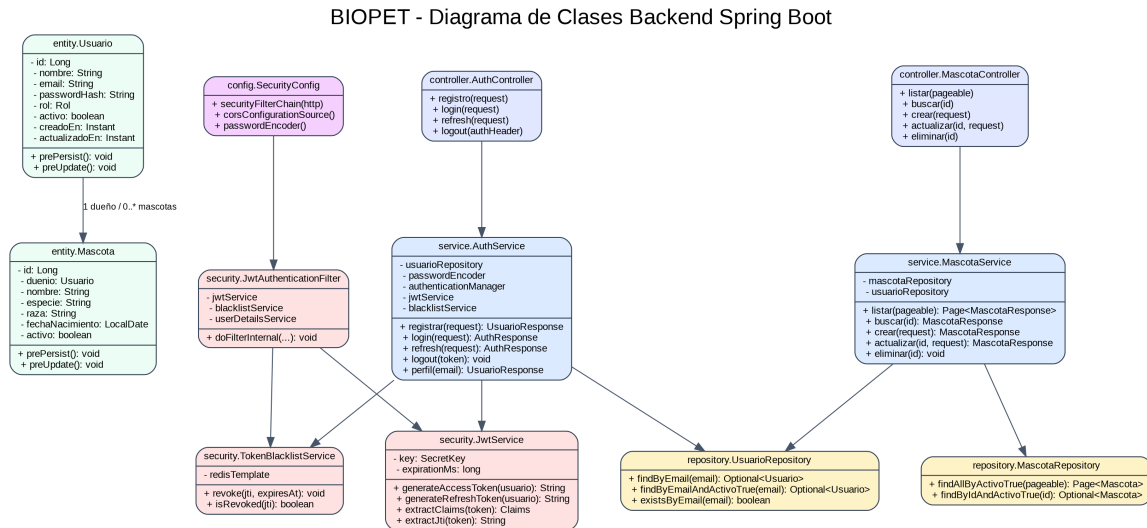


Figura 2: Diagrama de clases del backend Spring Boot.

Tabla de correspondencia Java ↔ PostgreSQL

Cuadro 1: Correspondencia de la entidad Usuario.

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y justificación
id	Long	BIGSERIAL	PK; secuencia; mejor rendimiento en <i>joins</i> que UUID.
nombre	String	VARCHAR(100)	NOT NULL.
email	String	VARCHAR(255)	UNIQUE NOT NULL; índice único.
passwordHash	String	VARCHAR(255)	BCrypt costo 12; nunca serializado (@JsonIgnore).
rol	enum Rol	VARCHAR(30)	CHECK; no se usa ENUM nativo de PostgreSQL.
creadoEn	Instant	TIMESTAMPTZ	UTC; @CreationTimestamp.
actualizadoEn	Instant	TIMESTAMPTZ	UTC; actualizado por <i>trigger</i> .
activo	boolean	BOOLEAN	<i>Soft delete</i> ; DEFAULT TRUE.

Cuadro 2: Correspondencia de la entidad Mascota.

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y justificación
id	Long	BIGSERIAL	PK; secuencia.
duenio	Usuario	BIGINT (FK)	@ManyToOne; FK duenio_id hacia usuarios(id).
nombre	String	VARCHAR(50)	NOT NULL.
especie	String	VARCHAR(30)	NOT NULL.
raza	String	VARCHAR(50)	NOT NULL.
fechaNacimiento	LocalDate	DATE	NOT NULL; @PastOrPresent en el DTO.
creadoEn	Instant	TIMESTAMPTZ	UTC.
actualizadoEn	Instant	TIMESTAMPTZ	UTC; <i>trigger</i> .
activo	boolean	BOOLEAN	<i>Soft delete</i> ; DEFAULT TRUE.

3.3. Diagrama de secuencia: flujo de autenticación JWT

El flujo de login comprende once pasos: (1) el cliente Angular envía POST /api/auth/login con {email, password}; (2) el AuthController delega en AuthenticationManager.authenticate(); (3) Spring Security consulta UserDetailsService.loadUserByUsername(); (4) este consulta UsuarioRepository.findByEmail(); (5) PostgreSQL devuelve el usuario e Hibernate lo mapea; (6) se verifica el hash BCrypt con PasswordEncoder.matches(); (7) JwtService firma el JWT con la clave secreta; (8) el servidor responde {accessToken, refreshToken, expiresIn}; (9) Angular guarda el *access token* en memoria y el *refresh token* en cookie HttpOnly; (10) en cada solicitud posterior el JwtInterceptor agrega Authorization: Bearer [token]; (11) el filtro JWT valida el token, consulta Redis y establece el SecurityContext.

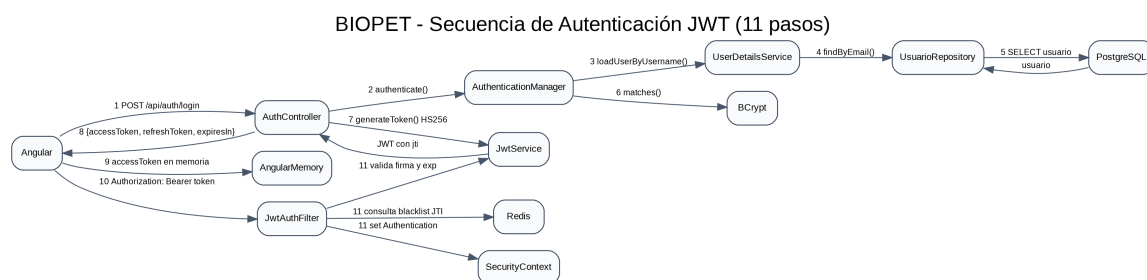


Figura 3: Diagrama de secuencia de la autenticación JWT.

3.4. Script Flyway de migración inicial

El esquema se crea con la migración V1__schema_inicial.sql, que emplea BIGSERIAL, TIMESTAMPTZ en UTC, restricciones CHECK y UNIQUE, una clave foránea y un *trigger* que actualiza automáticamente la columna actualizado_en.


```

1  CREATE TABLE IF NOT EXISTS usuarios (
2      id BIGSERIAL PRIMARY KEY,
3      nombre VARCHAR(100) NOT NULL,
4      email VARCHAR(255) NOT NULL,
5      password_hash VARCHAR(255) NOT NULL,
6      rol VARCHAR(30) NOT NULL DEFAULT 'ROLE_DUENO',
7      activo BOOLEAN NOT NULL DEFAULT TRUE,
8      creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
9      actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
10 );
11 CREATE UNIQUE INDEX IF NOT EXISTS idx_usuarios_email ON usuarios (
12     email);
13 ALTER TABLE usuarios ADD CONSTRAINT chk_usuarios_rol
14     CHECK (rol IN ('ROLE_ADMIN', 'ROLE_VETERINARIO', 'ROLE_DUENO', '
15         ROLE_AUXILIAR'));
16
17 CREATE TABLE IF NOT EXISTS mascotas (
18     id BIGSERIAL PRIMARY KEY,
19     duenio_id BIGINT NOT NULL,
20     nombre VARCHAR(50) NOT NULL,
21     especie VARCHAR(30) NOT NULL,
22     raza VARCHAR(50) NOT NULL,
23     fecha_nacimiento DATE NOT NULL,
24     activo BOOLEAN NOT NULL DEFAULT TRUE,
25     creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
26     actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
27     CONSTRAINT fk_mascotas_duenio FOREIGN KEY (duenio_id) REFERENCES
28         usuarios(id)
29 );
30
31 CREATE OR REPLACE FUNCTION set_actualizado_en()
32 RETURNS TRIGGER AS $$
33 BEGIN
34     NEW.actualizado_en = NOW();
35     RETURN NEW;
36 END;
37 $$ LANGUAGE plpgsql;
38
39 CREATE TRIGGER trg_usuarios_actualizado_en
40 BEFORE UPDATE ON usuarios
41 FOR EACH ROW EXECUTE FUNCTION set_actualizado_en();

```

Listing 1: database/migrations/V1__schema_inicial.sql (extracto).

Diagrama Entidad-Relación (complementario)

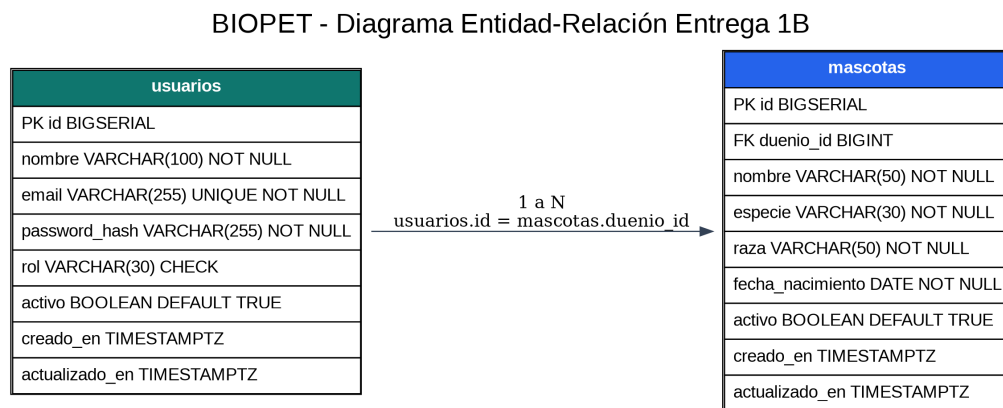


Figura 4: Diagrama entidad-relación de las tablas implementadas.

Diccionario de datos (complementario)

Cuadro 3: Diccionario de datos — tabla **usuarios**.

Columna	Tipo	Nulo	Defecto	Descripción
id	BIGSERIAL	No	secuencia	Clave primaria.
nombre	VARCHAR(100)	No	—	Nombres y apellidos.
email	VARCHAR(255)	No	—	Correo de acceso (índice único).
password_hash	VARCHAR(255)	No	—	Hash BCrypt; no serializado.
rol	VARCHAR(30)	No	ROLE_DUEÑO	Rol; restricción CHECK.
activo	BOOLEAN	No	TRUE	Borrado lógico.
creado_en	TIMESTAMPZ	No	NOW()	Fecha de creación (UTC).
actualizado_en	TIMESTAMPZ	No	NOW()	Última modificación (trigger).

Cuadro 4: Diccionario de datos — tabla **mascotas**.

Columna	Tipo	Nulo	Defecto	Descripción
id	BIGSERIAL	No	secuencia	Clave primaria.
dueño_id	BIGINT	No	—	FK hacia usuarios(id).

Columna	Tipo	Nulo	Defecto	Descripción
nombre	VARCHAR(50)	No	—	Nombre de la mascota.
especie	VARCHAR(30)	No	—	Especie (canino, felino, etc.).
raza	VARCHAR(50)	No	—	Raza de la mascota.
fecha_nacimiento	DATE	No	—	Fecha de nacimiento.
activo	BOOLEAN	No	TRUE	Borrado lógico.
creado_en	TIMESTAMP	No	NOW()	Fecha de creación (UTC).
actualizado_en	TIMESTAMP	No	NOW()	Última modificación (trigger).

4. Implementación

4.1. Estructura del repositorio (monorepo)

```

1 biopet/
2 |-- backend/
3 |   |-- src/main/java/com/biopet/
4 |   |   |-- config/           OpenApiConfig, SecurityConfig,
4 |   |   |   DataInitializer
5 |   |   |-- controller/      AuthController, UsuarioController,
5 |   |   |   MascotaController
6 |   |   |-- dto/             records de entrada/salida (sin @Entity)
7 |   |   |-- entity/          Usuario, Mascota, Rol
8 |   |   |-- exception/       GlobalExceptionHandler y excepciones de
8 |   |   |   dominio
9 |   |   |-- repository/      UsuarioRepository, MascotaRepository
10 |   |   |-- security/        JwtService, JwtAuthenticationFilter,
10 |   |   |   TokenBlacklistService
11 |   |   |-- service/         AuthService, MascotaService,
11 |   |   |   UserDetailsServiceImpl
12 |   |-- src/main/resources/
13 |   |   |-- application.yml
14 |   |   |-- db/migration/V1__schema_inicial.sql
15 |   |-- src/test/           AuthControllerTest + application-test.yml
16 |   |-- Dockerfile
17 |   |-- pom.xml
18 |-- frontend/               Angular (core/, features/) + Nginx +
18 |   Dockerfile
19 |-- database/migrations/     Copia del script Flyway
    
```

```

20 |-- docs/                                adr/, diagrams/, diccionario_datos.md,
    postman/
21 |-- docker-compose.yml                  Servicios: backend, postgres, redis,
    frontend
22 |-- .env.example                        DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET,
    ...
23 '-- .github/workflows/ci.yml

```

Listing 2: Árbol de directorios del repositorio.

4.2. Configuración de Spring Boot

```

1  spring:
2    datasource:
3      url: ${DB_URL:jdbc:postgresql://localhost:5432/biopet_db}
4      username: ${DB_USER:biopet_user}
5      password: ${DB_PASSWORD:biopet_pass}
6      driver-class-name: org.postgresql.Driver
7    jpa:
8      hibernate:
9        ddl-auto: validate                # Flyway gestiona el DDL
10     open-in-view: false                 # evitar LazyInitializationException
11     flyway:
12       enabled: true
13       locations: classpath:db/migration
14       baseline-on-migrate: true
15     data:
16       redis:
17         host: ${REDIS_HOST:localhost}
18         port: ${REDIS_PORT:6379}
19
20     security:
21       jwt:
22         secret: ${JWT_SECRET}            # minimo 256 bits
23         expiration-ms: ${JWT_EXPIRATION_MS:3600000} # 1 hora
24         refresh-expiration-ms: ${JWT_REFRESH_EXPIRATION_MS:604800000} #
           7 dias
25
26     springdoc:
27       api-docs: { path: /api/docs }
28       swagger-ui: { path: /api/swagger-ui.html, operationsSorter: method
           }
29
30     management:
31       endpoints:
32         web:
33           exposure:
34             include: health,info,metrics

```

Listing 3: backend/src/main/resources/application.yml (extracto).

4.3. Implementación del filtro JWT

El `JwtAuthenticationFilter` extiende `OncePerRequestFilter`. Extrae el token del encabezado `Authorization: Bearer [token]`, valida su firma y expiración con `JwtService`, verifica que sea un *access token* y que su JTI no figure en la lista negra de Redis, y finalmente establece un `UsernamePasswordAuthenticationToken` en el `SecurityContextHolder`.

```

1 String header = request.getHeader("Authorization");
2 if (header == null || !header.startsWith("Bearer ")) {
3     filterChain.doFilter(request, response);
4     return;
5 }
6 String token = header.substring(7);
7 try {
8     String email = jwtService.extractEmail(token);
9     String jti = jwtService.extractJti(token);
10    boolean valido = jwtService.isAccessToken(token) && !
        blacklistService.isRevoked(jti);
11    if (email != null && valido
12        && SecurityContextHolder.getContext().getAuthentication()
            == null) {
13        UserDetails user = userDetailsService.loadUserByUsername(
            email);
14        var authToken = new UsernamePasswordAuthenticationToken(
            user, null, user.getAuthorities());
15        SecurityContextHolder.getContext().setAuthentication(
            authToken);
16    }
17 } catch (JwtException | IllegalArgumentException ex) {
18     SecurityContextHolder.clearContext();
19 }
20 filterChain.doFilter(request, response);
21

```

Listing 4: `JwtAuthenticationFilter` (núcleo del método `doFilterInternal`).

4.4. Endpoints implementados

Cuadro 5: Endpoints disponibles en la Entrega 1B.

Método	URL	Auth	Descripción
POST	/api/auth/registro	No	Registrar usuario (rol <code>ROLE_DUENO</code>); 201.
POST	/api/auth/login	No	Autenticar; devuelve <i>access</i> y <i>refresh token</i> .
POST	/api/auth/logout	JWT	Agrega el JTI a la lista negra de Redis; 204.
POST	/api/auth/refresh	Refresh	Emite un nuevo <i>access token</i> .

Método	URL	Auth	Descripción
GET	/api/usuarios/me	JWT	Perfil del usuario autenticado.
GET	/api/mascotas	JWT	Listar con paginación (?page=0&size=10&sort=id,asc).
GET	/api/mascotas/{id}	JWT	Buscar por ID; 404 si no existe.
POST	/api/mascotas	JWT (ADMIN/VET/AUX)	Crear; valida con @Valid; 201.
PUT	/api/mascotas/{id}	JWT (ADMIN/VET/AUX)	Actualizar; 200.
DELETE	/api/mascotas/{id}	JWT (ADMIN/VET/AUX)	Soft delete (activo=false); 204.
GET	/api/docs	No	Swagger UI (Springdoc OpenAPI).

4.5. Estructura del JWT emitido

Cuadro 6: *Claims* del token emitido por el sistema.

Claim	Valor de ejemplo	Descripción
sub	"42"	ID del usuario (sujeto del token).
email	"juan@uteq.edu.ec"	Correo del usuario.
rol	"ROLE_DUEÑO"	Rol de Spring Security.
typ	"access"	Tipo de token (access o refresh).
iat	1735689600	Emitido en (<i>timestamp</i> Unix).
exp	1735693200	Expiración (iat + 3600 s).
jti	"550e8400-e29b..."	ID único; usado para la lista negra en Redis.

4.6. Implementación Angular: interceptor JWT

El *access token* se almacena **en memoria**, en una variable privada del servicio `AuthService` —nunca en `localStorage`—. El `JwtInterceptor` clona cada solicitud y le agrega el encabezado `Authorization`, mientras que el `AuthGuard` protege las rutas con `canActivate` y redirige al *login* cuando no hay sesión.

```

1 // jwt.interceptor.ts
2 export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
3   const token = inject(AuthService).getToken();
4   if (!token) return next(req);
5   return next(req.clone({ setHeaders: { Authorization: `Bearer ${
6     token}` } })));
7 }
8 // auth.guard.ts
9 export const authGuard: CanActivateFn = () => {
10   const auth = inject(AuthService); const router = inject(Router);
11   if (auth.isLoggedIn()) return true;
12   router.navigate(['/login']); return false;
13 };
    
```

```

14
15 // auth.service.ts (el token vive en memoria, no en localStorage)
16 private accessToken: string | null = null;
17 getToken(): string | null { return this.accessToken; }

```

Listing 5: frontend/src/app/core/ — interceptor, guard y servicio (extracto).

5. Pruebas

5.1. Pruebas unitarias del servicio de autenticación

Se implementaron cinco pruebas con JUnit 5 y MockMvc, ejecutadas sobre H2 en modo de compatibilidad con PostgreSQL (sin contenedores).

Cuadro 7: Casos de prueba automatizados.

#	Caso de prueba	Método	Esperado	Resultado
1	Login con credenciales correctas	loginExitoso()	200 + JWT	PASS
2	Login con contraseña incorrecta	loginClaveIncorrecta()	401	PASS
3	Registro con email existente	registroEmailDuplicado()	409	PASS
4	Acceso protegido sin token	accesoSinToken()	401	PASS
5	Acceso protegido con token válido	accesoConTokenValido()	200	PASS

```

1 @Test
2 void accesoSinToken() throws Exception {
3     mockMvc.perform(get("/api/usuarios/me"))
4         .andExpect(status().isUnauthorized());
5 }

```

Listing 6: Ejemplo: acceso sin token debe devolver 401.

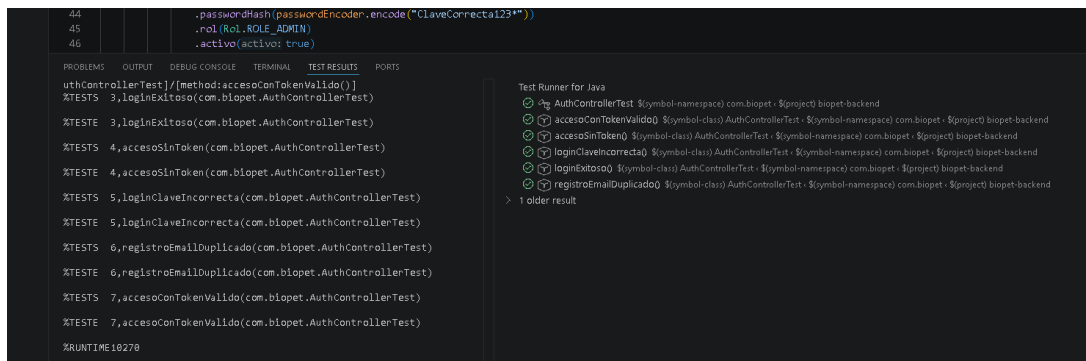


Figura 5: Ejecución de la batería de pruebas unitarias en verde.

5.2. Pruebas de integración con Postman

Las siguientes capturas demuestran el funcionamiento de la API de extremo a extremo.

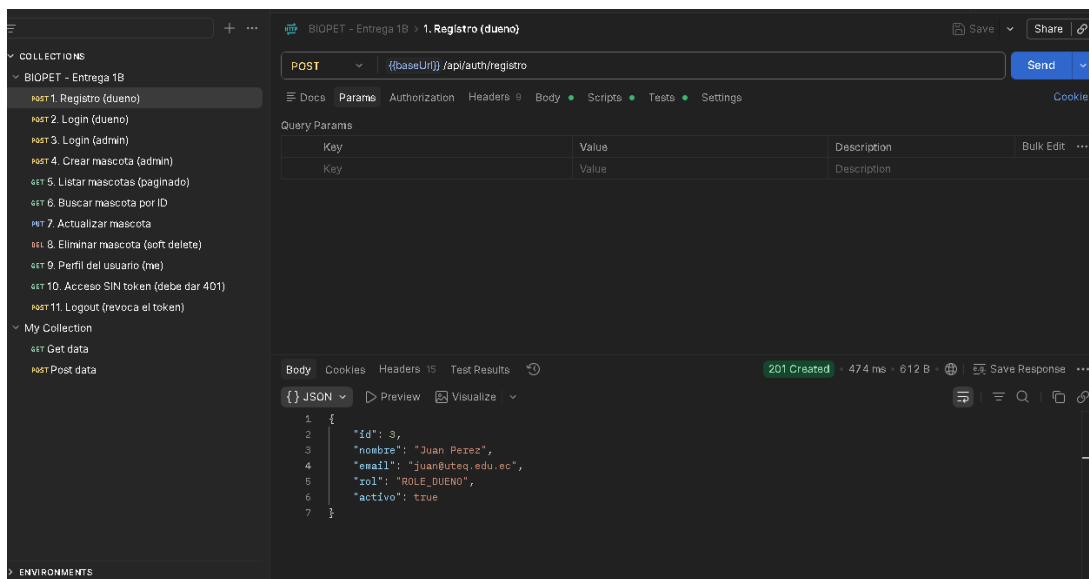


Figura 6: Registro de un nuevo usuario.

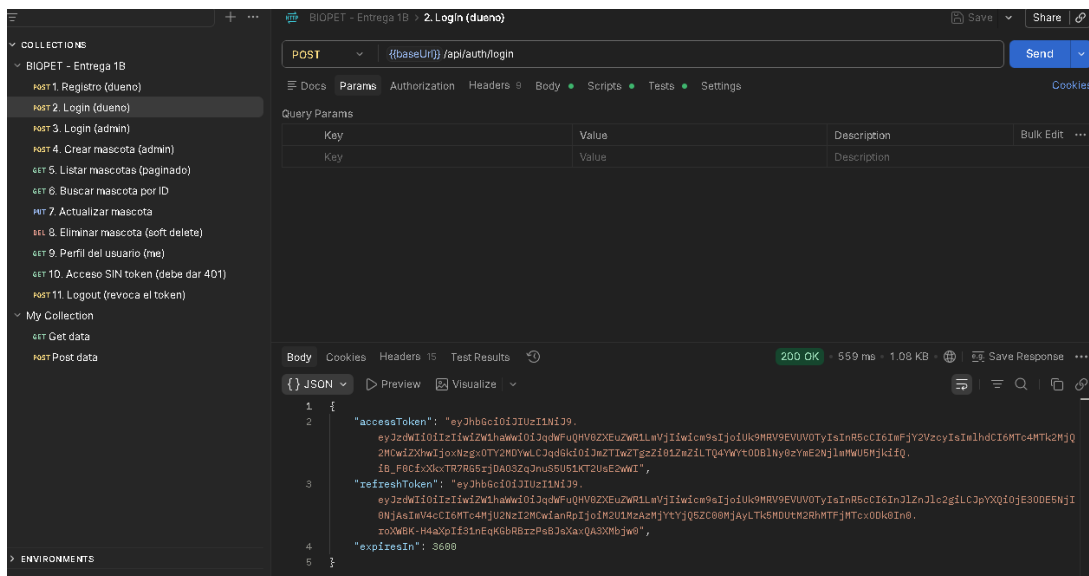


Figura 7: Inicio de sesión exitoso con el JWT visible.

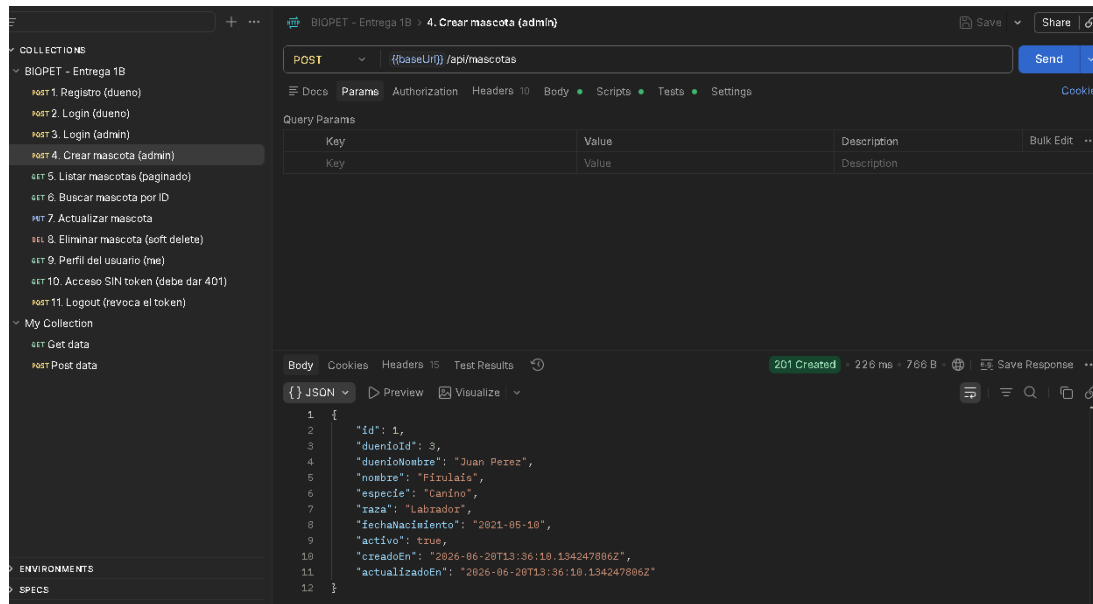


Figura 8: CRUD- Crear Mascota con *Bearer token*.

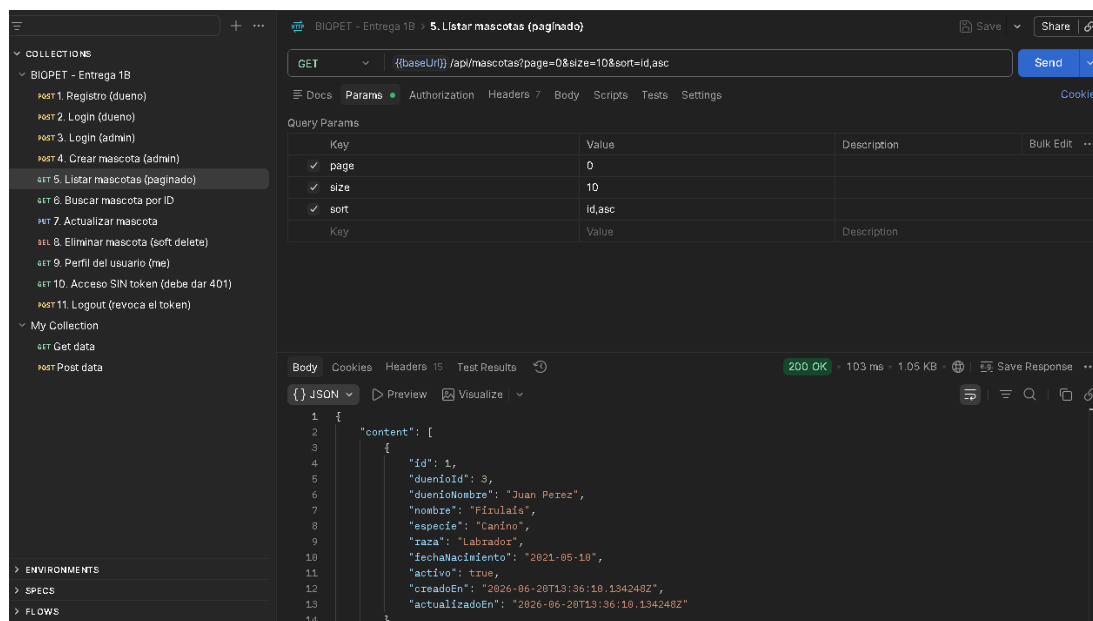


Figura 9: CRUD- Listar Mascota (Paginado) con *Bearer token*.

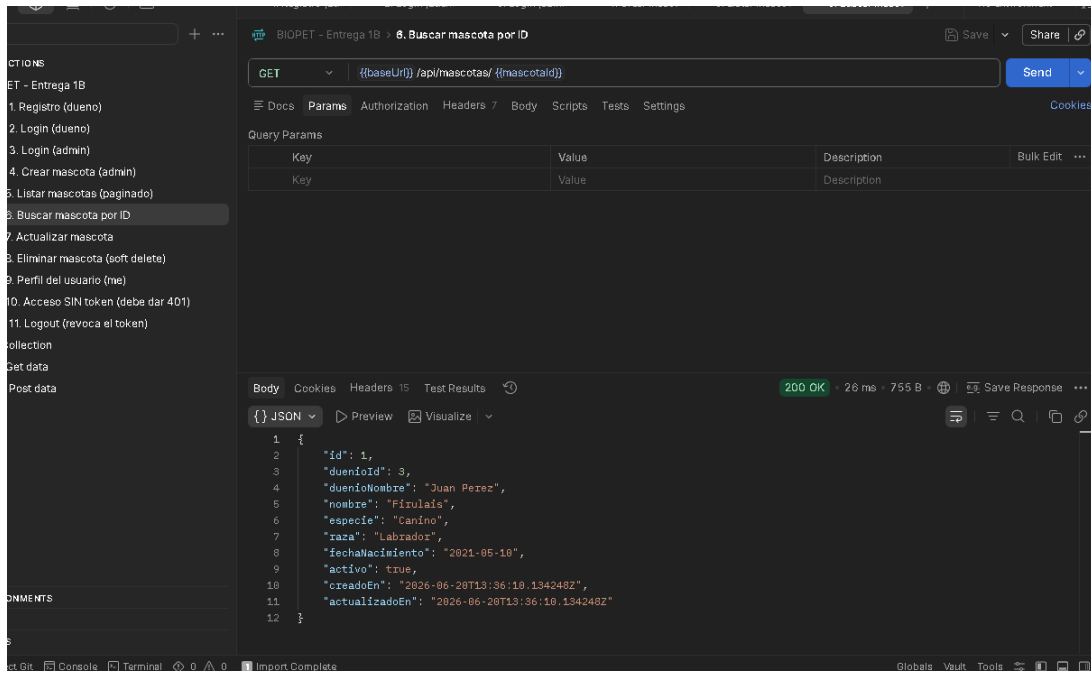


Figura 10: CRUD- Buscar Mascota por ID con *Bearer token*.

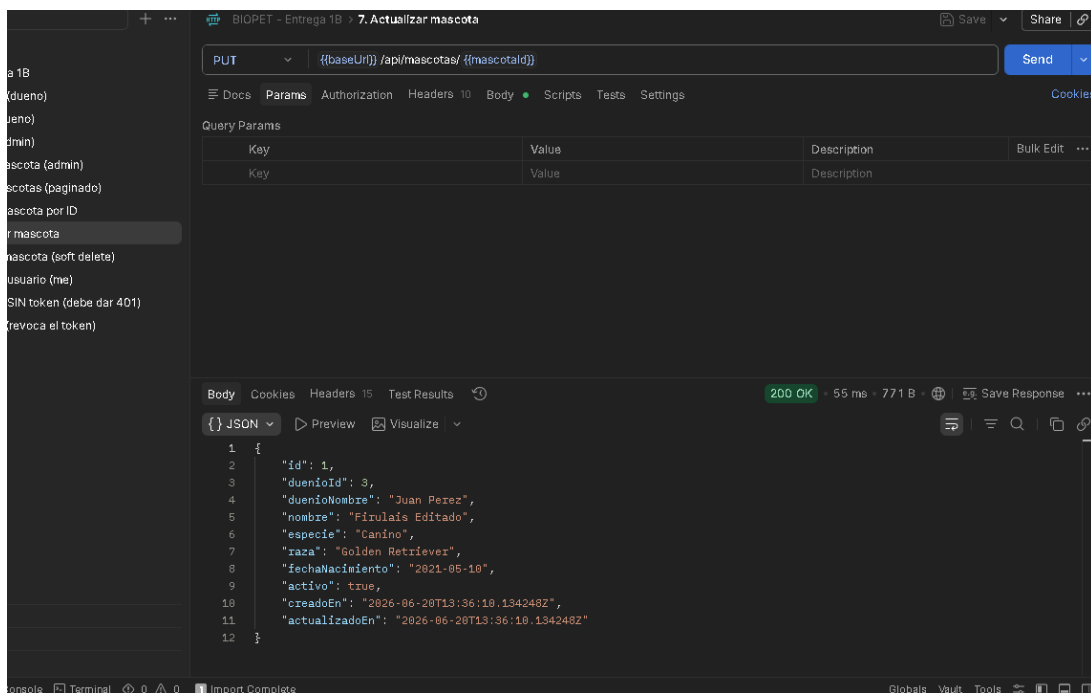


Figura 11: CRUD- Actualizar Mascota con *Bearer token*.

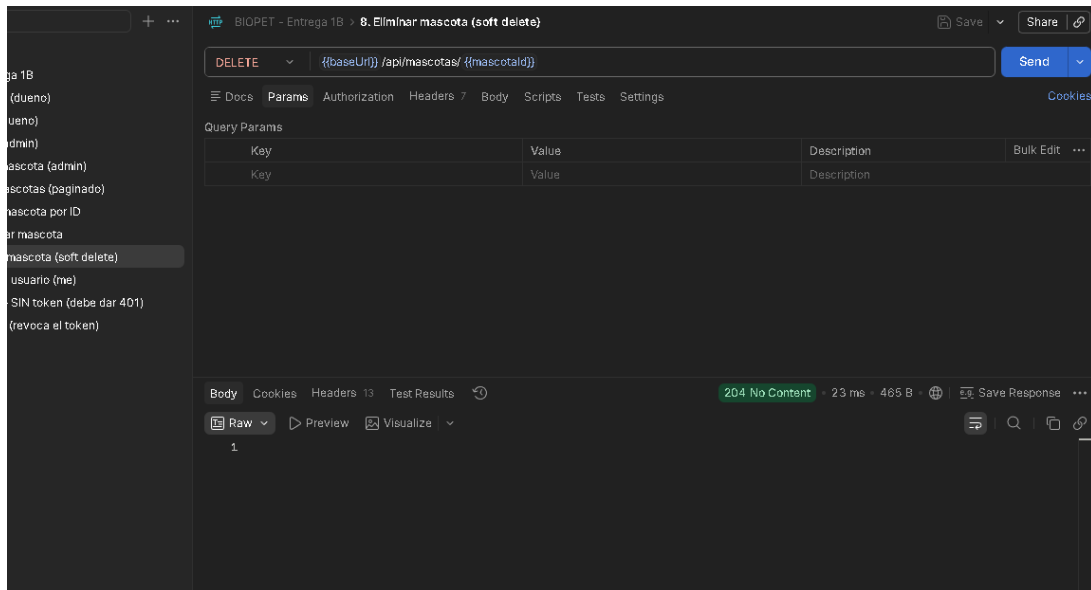


Figura 12: CRUD- Eliminar Mascota con *Bearer token*.

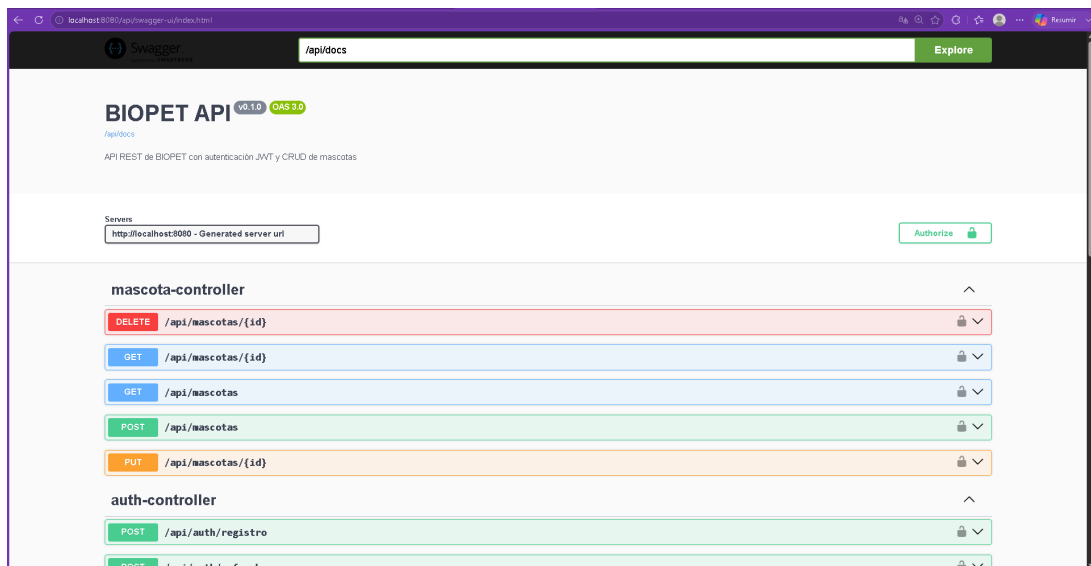


Figura 13: Documentación interactiva de la API en Swagger UI.

5.3. Métricas de rendimiento

Se registran en la Tabla 8 los tiempos reales medidos en nuestro equipo (historial de Postman, JMeter o k6).

Cuadro 8: Métricas de rendimiento de los endpoints.

Endpoint	Prom. (ms)	P95 (ms)	Observación
POST /api/auth/login	230 ms	295 ms	Incluye verificación BCrypt (costo 12).
GET /api/mascotas (50 registros)	23 ms	28 ms	Sin caché.
POST /api/mascotas	17 ms	18 ms	Con validación y <i>commit</i> .
GET /api/mascotas con caché Redis	15 ms	18 ms	Comparar con la fila sin caché.

6. Seguridad Implementada

Se documentan los controles de seguridad activos, cada uno alineado con una categoría del OWASP Top 10 y con su evidencia correspondiente.

Cuadro 9: Controles de seguridad implementados.

Control	Implementación	OWASP	Evidencia
Hash de contraseñas	BCrypt costo 12 vía BCryptPasswordEncoder	A02	Captura de BD: hash \$2a\$12\$...
Tokens JWT de corta vida	<i>access</i> 1 h; <i>refresh</i> 7 d	A07	<i>Payload</i> decodificado
Lista negra de tokens	JTI revocado en Redis con TTL	A07	Prueba de <i>logout</i> + intento de reuso
Cabeceras de seguridad	X-Content-Type-Options, X-Frame-Options, CSP	A05	<code>curl -I</code> al endpoint
CORS configurado	Orígenes permitidos explícitamente en SecurityConfig	A05	Prueba desde dominio no permitido
Sin SQL directo	Spring Data JPA + Flyway; cero concatenación	A03	Revisión de código
Roles de acceso	@PreAuthorize por rol	A01	Prueba de acceso denegado (403)

```

PS C:\Users\jaime\Downloads\BIOPET-Entrega1B\BIOPET-Entrega1B> docker exec -it biopet-postgres psql -U biopet_user -d biopet_db -c "SELECT id, email, password_hash, rol FROM usuarios;"
id | email | password_hash | rol
-----
1 | jaime@biopet.com | $2a$12$5E4IEy.UWLMFie7WpQxwexBzWpVd4fy3Me/jnUk9/hSK.vjYopp6 | ROLE_ADMIN
2 | admin@biopet.ec | $2a$12$xp2zRaClb/Zfh3mVfUe57Ujg81FxaUwL8/WffjZoc/M.tMm | ROLE_ADMIN
3 | juan@uteq.edu.ec | $2a$12$MNgyoqZ5N0puH7nBE3KS.Oyga0R61ZEpb3K1m8UIGFEVAt4cLfs | ROLE_DUEÑO
(3 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug biopet-postgres
Learn more at: https://docs.docker.com/go/debug-cil/
PS C:\Users\jaime\Downloads\BIOPET-Entrega1B\BIOPET-Entrega1B>
    
```

Figura 14: Evidencia de almacenamiento seguro de contraseñas (A02).

```

PS C:\Users\jaime\Downloads\BIOPET-Entrega1B\BIOPET-Entrega1B> curl.exe -i http://localhost:8080/actuator/health
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: default-src 'self'; frame-ancestors 'self'; object-src 'none'
Content-Type: application/vnd.spring-boot.actuator.v3+json
Transfer-Encoding: chunked
Date: Sat, 20 Jun 2020 13:43:58 GMT

{"status":"UP"}
PS C:\Users\jaime\Downloads\BIOPET-Entrega1B\BIOPET-Entrega1B>
    
```

Figura 15: Cabeceras de seguridad en la respuesta HTTP (A05).

7. Docker Compose y Despliegue

El sistema se orquesta en cuatro servicios (postgres, redis, backend y frontend), cada uno con su *health check* y con dependencias condicionadas al estado saludable de los servicios de los que depende mediante `depends_on`.

```

1  services:
2    postgres:
3      image: postgres:16-alpine
4      container_name: biopet-postgres
5      environment:
6        POSTGRES_DB: ${DB_NAME:-biopet_db}
7        POSTGRES_USER: ${DB_USER:-biopet_user}
8        POSTGRES_PASSWORD: ${DB_PASSWORD:-biopet_pass}
9      ports:
10       - "5432:5432"
11      volumes:
12       - postgres_data:/var/lib/postgresql/data
13      healthcheck:
14        test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-biopet_user} -d $
15              {DB_NAME:-biopet_db}"]
16        interval: 10s
17        timeout: 5s
18        retries: 5
19
20    redis:
21      image: redis:7-alpine
22      container_name: biopet-redis
23      ports:
24       - "6379:6379"
25      healthcheck:
26        test: ["CMD", "redis-cli", "ping"]
27        interval: 10s
28        timeout: 5s
29        retries: 5
    
```

```

30  backend:
31      build: ./backend
32      container_name: biopet-backend
33      env_file: .env
34      ports:
35          - "8080:8080"
36      depends_on:
37          postgres:
38              condition: service_healthy
39          redis:
40              condition: service_healthy
41      healthcheck:
42          test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/
43              health | grep UP || exit 1"]
44          interval: 20s
45          timeout: 10s
46          retries: 10
47
48  frontend:
49      build: ./frontend
50      container_name: biopet-frontend
51      ports:
52          - "4200:80"
53      depends_on:
54          backend:
55              condition: service_healthy
56      healthcheck:
57          test: ["CMD-SHELL", "wget -qO- http://localhost/ || exit 1"]
58          interval: 20s
59          timeout: 10s
60          retries: 5
61
62  volumes:
63      postgres_data:
    
```

Listing 7: docker-compose.yml

```

PS C:\Users\jaime\Downloads\BIOPET-Entrega1B\BIOPET-Entrega1B> docker compose up --build -d

#32 [frontend] resolving provenance for metadata file
#32 DONE 0.0s

#33 [backend] resolving provenance for metadata file
#33 DONE 0.0s

[+] up 6/6
✓ Image biopet-entregaib-frontend Built
✓ Image biopet-entregaib-backend Built
✓ Container biopet-backend Healthy
✓ Container biopet-frontend Started
✓ Container biopet-redis Healthy
✓ Container biopet-postgres Healthy

What's next:
Filter, search, and stream logs from all your Compose services
in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=biopet-entregaib
● PS C:\Users\jaime\Downloads\BIOPET-Entrega1B\BIOPET-Entrega1B> docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
biopet-backend	biopet-entregaib-backend	"java -jar /app/app..."	backend	46 seconds ago	Up 34 seconds (healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
biopet-frontend	biopet-entregaib-frontend	"docker-entrypoint..."	frontend	45 seconds ago	Up 13 seconds (health: starting)	0.0.0.0:4200->80/tcp, [::]:4200->80/tcp
biopet-postgres	postgres:16-alpine	"docker-entrypoint.s..."	postgres	9 hours ago	Up 44 seconds (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
biopet-redis	redis:7-alpine	"docker-entrypoint.s..."	redis	9 hours ago	Up 44 seconds (healthy)	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp

Figura 16: Servicios en ejecución tras docker compose up.

8. Historial de Git

El desarrollo sigue el estándar de *Conventional Commits* y una estrategia de ramas: las funcionalidades se desarrollaron en `feature/autenticacion-jwt` y `feature/crud-mascota`, integradas a `develop`. La entrega se marca con la etiqueta `v0.1.0-entrega-1b`. El historial debe evidenciar *commits* de los tres integrantes.

Ejemplos de *commits*: `feat(auth): emitir JWT con HS256`; `feat(mascota): CRUD con borrado lógico y paginación`; `fix(security): devolver 401 ante acceso sin token`; `test(auth): pruebas de autenticación`; `docs(adr): justificar Redis para blacklist (ADR-003)`.

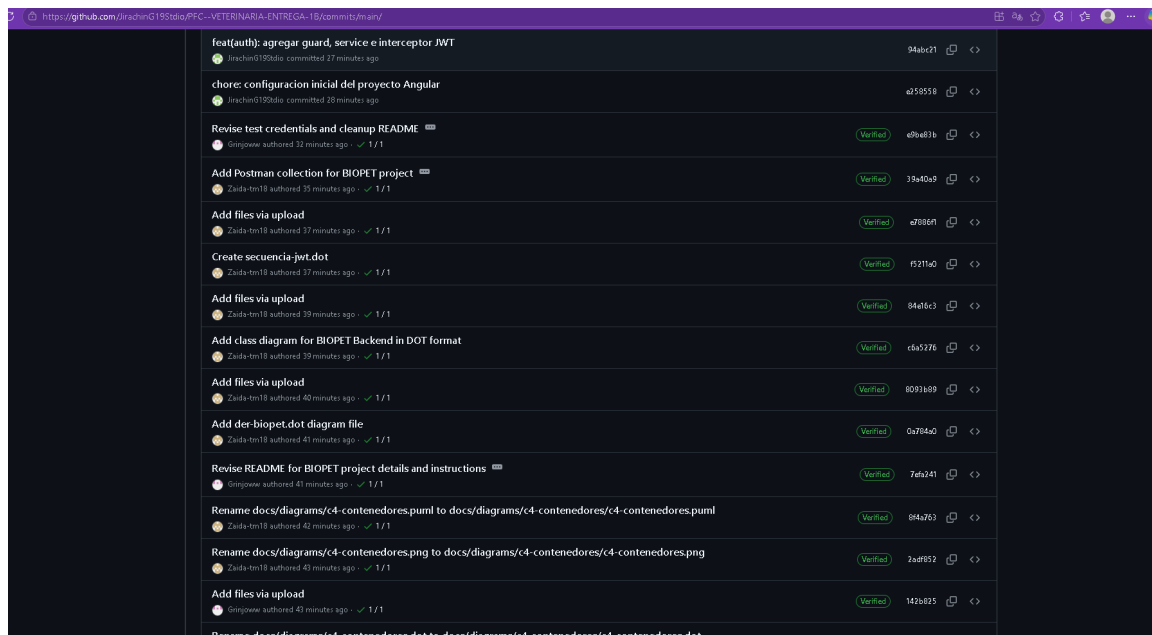


Figura 17: Historial de *commits* con Conventional Commits, ramas y *tag* de entrega.

9. Conclusiones

Objetivo 1 — Autenticación *stateless* con JWT. El objetivo se logró por completo: el sistema implementa registro, inicio de sesión, cierre de sesión y renovación de token, emitiendo *access* y *refresh tokens* firmados con HS256 mediante `jjwt 0.12`. El mayor desafío técnico fue conciliar la naturaleza *stateless* de JWT con la necesidad de revocar sesiones, pues el servidor no almacena los tokens emitidos. A ello se sumó que, inicialmente, el acceso a un recurso protegido sin token devolvía un código 403 en lugar del 401 esperado, debido al punto de entrada por defecto de Spring Security. Ambos problemas se resolvieron: la revocación, mediante una lista negra de identificadores JTI en Redis con tiempo de vida igual a la expiración del token, consultada por el filtro en cada solicitud; y el código de estado, configurando explícitamente un `authenticationEntryPoint` que responde 401 y un `accessDeniedHandler` que responde 403, dejando el comportamiento alineado con las buenas prácticas REST.

Objetivo 2 — CRUD con Spring Data JPA y Flyway. Se alcanzó el objetivo implementando el ciclo completo de la entidad Mascota con paginación, borrado lógico y gestión del esquema mediante migraciones versionadas, sin una sola concatenación de SQL. El principal desafío fue una excepción de inicialización perezosa (`LazyInitializationException`) al serializar la mascota junto con su dueño, consecuencia de haber deshabilitado `open-in-view` para mejorar el rendimiento. Se resolvió mapeando las entidades a objetos de transferencia (DTO) *dentro* de la transacción de servicio, de modo que las relaciones perezosas se resuelven mientras la sesión de Hibernate sigue abierta. Adicionalmente, mantener `ddl-auto` en modo `validate` obligó a que las entidades y el script Flyway coincidieran exactamente en nombres de columna y tipos, lo que reforzó la consistencia del modelo.

Objetivo 3 — Seguridad OWASP y pruebas. El objetivo se cumplió: se documentaron siete controles alineados con el OWASP Top 10 y se implementaron cinco pruebas unitarias que cubren los flujos críticos de autenticación y autorización. El desafío más relevante fue de diseño de seguridad: el endpoint público de registro permitía, en su primera versión, que cualquier solicitante se asignara el rol de administrador, configurando una escalada de privilegios. Se resolvió forzando el rol básico `ROLE_DUENO` en el registro público e ignorando cualquier rol enviado por el cliente. En el plano de las pruebas, fue necesario configurar un perfil dedicado con base de datos H2 y caché en memoria para que la batería se ejecutara de forma aislada, sin requerir PostgreSQL ni Redis activos.

Objetivo 4 — Despliegue con Docker y evaluación de la caché. Se logró orquestar los cuatro servicios con un único comando y se evaluó el efecto de la caché en las consultas de lectura. El mayor desafío surgió al cachear en Redis el resultado paginado del listado de mascotas: el serializador por defecto exigía que el objeto de transferencia fuera serializable, lo que provocaba un error en tiempo de ejecución. Se resolvió haciendo que el DTO implementara la interfaz `Serializable`. Asimismo, garantizar un arranque ordenado requirió configurar *health checks* y condicionar el inicio del backend al estado saludable de la base de datos y de Redis. **Proyección a la Entrega 2:** se desarrollará la interfaz de usuario completa en Angular consumiendo la API protegida, se ampliarán las entidades del dominio (citas e historiales clínicos) y se extenderá la cobertura de pruebas a las capas de servicio y persistencia.

Referencias

- [1] Walls, C. (2022). *Spring Boot in action* (2.^a ed.). Manning Publications. ISBN 978-1-617-29454-7.
- [2] Spilca, L. (2024). *Spring Security in action* (2.^a ed.). Manning Publications. ISBN 978-1-633-43797-5.
- [3] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. <https://tools.ietf.org/html/rfc7519>

- [4] Hardt, D. (2012). *The OAuth 2.0 Authorization Framework* (RFC 6749). IETF. <https://tools.ietf.org/html/rfc6749>
- [5] OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- [6] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].
- [7] Rahmatulloh, A., Gunawan, R., & Nugraha, I. P. (2019). Performance comparison of signed algorithms on JSON Web Token. *IOP Conference Series: Materials Science and Engineering*, 550, 012023.
- [8] Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the USENIX Annual Technical Conference* (pp. 81–91). USENIX.
- [9] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
- [10] Spring Framework Team. (2024). *Spring Security reference documentation*. <https://docs.spring.io/spring-security/reference/>
- [11] Hibernate Project, *Hibernate ORM Documentation*. [Online]. Available: <https://hibernate.org/orm/documentation/>
- [12] V. Bucko, K. Vishi, D. Krasniqi, and F. Rexha, “Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History,” 2023.
- [13] C. Boettiger, “An introduction to Docker for reproducible research,” 2014. [Online]. Available: <https://dl.acm.org/doi/10.1145/2723872>
- [14] M. Ben Fredj, O. Cheikhrouhou, M. Krichen, M. Hamam, and A. Derhab, “An OWASP Top Ten Driven Survey on Web Application Protection Methods,” 2020.